Hot 100 · 01-10（哈希 / 双指针 / 滑动窗口）

1. 两数之和（Two Sum，LC 1，Easy）

题意：给定整数数组 `nums` 和目标 `target`，返回和为 `target` 的两个下标。

考点：哈希表换时间；面试常追问「能否一次遍历」「有重复值怎么办」。

思路：遍历时用哈希表记录「值 → 下标」。对当前 `x`，查 `target - x` 是否已出现，命中即返回。一次遍历即可，避免 $O(n^2)$ 暴力。

```
def twoSum(nums: list[int], target: int) -> list[int]:
    seen = {}                        # 值 -> 下标
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []
```

复杂度：时间 O(n)，空间 O(n)。

最 Pythonic 写法：用 `dict.get` 配合海象运算符，一次查询替代「先判断再取值」。

```
def twoSum(nums: list[int], target: int) -> list[int]:
    seen = {}
    for i, x in enumerate(nums):
        if (j := seen.get(target - x)) is not None:
            return [j, i]
        seen[x] = i
    return []
```

2. 字母异位词分组（Group Anagrams，LC 49，Medium）

题意：把由相同字母重排得到的字符串归到一组。

考点：如何设计「异位词」的唯一 key；排序 key vs 计数 key 的取舍。

思路：异位词排序后相同，用排序串作 key 分组；若字符串很长，可用 26 长度的计数元组作 key，把单串 key 计算从 O(LlogL) 降到 O(L)。

```
from collections import defaultdict

def groupAnagrams(strs: list[str]) -> list[list[str]]:
    groups = defaultdict(list)
```

```
    for s in strs:
        cnt = [0] * 26
        for ch in s:
            cnt[ord(ch) - 97] += 1
        groups[tuple(cnt)].append(s)   # 计数元组作 key
    return list(groups.values())
```

复杂度：时间 O(n·L)，空间 O(n·L)。

最 Pythonic 写法：直接用排序后的字符串作 key，代码最短最直观（单串 key 计算 O(L·logL)）。

```
from collections import defaultdict

def groupAnagrams(strs: list[str]) -> list[list[str]]:
    groups = defaultdict(list)
    for s in strs:
        groups[''.join(sorted(s))].append(s)   # 排序串作 key
    return list(groups.values())
```

---

3. 最长连续序列（Longest Consecutive Sequence，LC 128，Medium）

题意：无序数组中找最长「连续整数」序列的长度，要求 O(n)。

考点：为什么不能排序（排序是 O(nlogn)）；如何用哈希集合保证每个数只被扩展一次。

思路：放入集合去重。只从「序列起点」（即 `x-1` 不在集合中）开始向右计数，保证整体均摊 O(n)。

```
def longestConsecutive(nums: list[int]) -> int:
    s = set(nums)
    best = 0
    for x in s:
        if x - 1 in s:            # 不是起点，跳过
            continue
        length = 1
        while x + length in s:    # 从起点向右扩
            length += 1
        best = max(best, length)
    return best
```

复杂度：时间 O(n)，空间 O(n)。

最 Pythonic 写法：排序去重后做一次「配对扫描」，逻辑最清晰（代价是 O(n·logn)，严格 O(n) 仍用上面的集合法）。

```
def longestConsecutive(nums: list[int]) -> int:
    if not nums:
```

```
        return 0
    uniq = sorted(set(nums))
    best = run = 1
    for prev, cur in zip(uniq, uniq[1:]):
        run = run + 1 if cur == prev + 1 else 1
        best = max(best, run)
    return best
```

---

4. 移动零（Move Zeroes，LC 283，Easy）

题意：把所有 0 移到末尾，保持非零元素相对顺序，原地操作。

考点：原地、稳定；双指针「快慢指针」模板。

思路：慢指针 `slow` 指向下一个非零应放的位置，快指针扫描，遇非零就交换并推进 `slow`。

```
def moveZeroes(nums: list[int]) -> None:
    slow = 0
    for fast in range(len(nums)):
        if nums[fast] != 0:
            nums[slow], nums[fast] = nums[fast], nums[slow]
            slow += 1
```

复杂度：时间 O(n)，空间 O(1)。

最 Pythonic 写法：列表推导挑出非零元素，再用切片赋值原地回填零，可读性最佳（额外 O(n) 临时空间）。

```
def moveZeroes(nums: list[int]) -> None:
    nonzero = [x for x in nums if x != 0]
    nums[:] = nonzero + [0] * (len(nums) - len(nonzero))   # 原地回填
```

---

5. 盛最多水的容器（Container With Most Water，LC 11，Medium）

题意：每个柱子高 `height[i]`，选两条线与 x 轴围成容器，求最大容水量。

考点：贪心 + 对撞双指针的正确性证明（为何移动较矮的一侧）。

思路：左右指针向内收缩，面积 = `min(左,右) * 宽`。移动较矮一侧才可能变大，移动较高一侧只会变差。

```
def maxArea(height: list[int]) -> int:
    l, r = 0, len(height) - 1
    best = 0
    while l < r:
        best = max(best, min(height[l], height[r]) * (r - l))
        if height[l] < height[r]:
```

```
            l += 1
        else:
            r -= 1
    return best
```

复杂度：时间 O(n)，空间 O(1)。

最 Pythonic 写法：对撞双指针本身已是最优解，这里把初始化合并成一行，仍是该题最 Pythonic 的形态。

```
def maxArea(height: list[int]) -> int:
    l, r, best = 0, len(height) - 1, 0
    while l < r:
        best = max(best, min(height[l], height[r]) * (r - l))
        if height[l] < height[r]:
            l += 1
        else:
            r -= 1
    return best
```

---

6. 三数之和（3Sum，LC 15，Medium）

题意：找出所有和为 0 且不重复的三元组。

考点：去重技巧（这是面试最易错点）；排序 + 双指针把 $O(n^3)$ 降到 $O(n^2)$。

思路：先排序，固定 `i`，在 `i` 右侧用对撞双指针找两数之和为 `-nums[i]`。三处跳过重复值。

```
def threeSum(nums: list[int]) -> list[list[int]]:
    nums.sort()
    n, res = len(nums), []
    for i in range(n - 2):
        if nums[i] > 0:
            break                       # 后面都更大，不可能为 0
        if i > 0 and nums[i] == nums[i - 1]:
            continue                    # 固定值去重
        l, r = i + 1, n - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0:
                l += 1
            elif s > 0:
                r -= 1
            else:
                res.append([nums[i], nums[l], nums[r]])
                l += 1
                r -= 1
                while l < r and nums[l] == nums[l - 1]:
                    l += 1              # 左指针去重
                while l < r and nums[r] == nums[r + 1]:
```

```
                    r -= 1                  # 右指针去重
    return res
```

复杂度：时间 $O(n^2)$，空间 O(1)（不计结果）。

最 Pythonic 写法：固定一个数后用哈希集合找另外两数，借助 `set` 自动去重，省去三处手动跳过（$O(n^2)$ 时间、O(n) 空间）。

```
def threeSum(nums: list[int]) -> list[list[int]]:
    nums.sort()
    res = set()
    for i in range(len(nums) - 2):
        seen = set()
        for x in nums[i + 1:]:
            need = -nums[i] - x
            if need in seen:
                res.add((nums[i], need, x))
            seen.add(x)
    return [list(t) for t in res]
```

---

7. 接雨水（Trapping Rain Water，LC 42，Hard）

题意：给柱状高度数组，求下雨后能接的雨水总量。

考点：经典 Hard；至少要会「双指针 O(1) 空间」，能讲清「短板决定水位」。

思路：每个位置的水量 = `min(左侧最高, 右侧最高) - 当前高度`。双指针维护 `left_max / right_max`，哪边矮就结算哪边。

```
def trap(height: list[int]) -> int:
    l, r = 0, len(height) - 1
    left_max = right_max = 0
    water = 0
    while l < r:
        if height[l] < height[r]:
            left_max = max(left_max, height[l])
            water += left_max - height[l]
            l += 1
        else:
            right_max = max(right_max, height[r])
            water += right_max - height[r]
            r -= 1
    return water
```

复杂度：时间 O(n)，空间 O(1)。

最 Pythonic 写法：用 `accumulate(max)` 求出每个位置的左、右最高柱，再一行求和（O(n) 额外空间换极简表达）。

```python
from itertools import accumulate

def trap(height: list[int]) -> int:
    if not height:
        return 0
    left = accumulate(height, max)
    right = list(accumulate(reversed(height), max))[::-1]
    return sum(min(l, r) - h for l, r, h in zip(left, right, height))
```

---

8. 无重复字符的最长子串（Longest Substring Without Repeating Characters，LC 3，Medium）

题意：求不含重复字符的最长子串长度。

考点：滑动窗口模板；记录字符最近出现位置加速左指针跳转。

思路：右指针扩张，遇到窗口内已出现的字符，把左指针跳到该字符上次位置 +1。用字典记录每个字符最后下标。

```python
def lengthOfLongestSubstring(s: str) -> int:
    last = {}        # 字符 -> 最近一次出现的下标
    left = 0
    best = 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] >= left:
            left = last[ch] + 1
        last[ch] = right
        best = max(best, right - left + 1)
    return best
```

复杂度：时间 O(n)，空间 O(min(n, 字符集))。

最 Pythonic 写法：用集合维护当前窗口，遇重复就从左收缩，语义比记录下标更直白。

```python
def lengthOfLongestSubstring(s: str) -> int:
    window = set()
    left = best = 0
    for ch in s:
        while ch in window:
            window.discard(s[left])
            left += 1
        window.add(ch)
```

```
            best = max(best, len(window))
        return best
```

---

9. 找到字符串中所有字母异位词（Find All Anagrams in a String，LC 438，Medium）

题意：找出 `s` 中所有是 `p` 的异位词的子串起始索引。

考点：定长滑动窗口 + 计数数组比较；用「差异计数」做到 O(1) 比较。

思路：维护长度为 `len(p)` 的窗口频次，与 `p` 的频次比较。可用整体计数数组并比较，简单直观。

```
from collections import Counter

def findAnagrams(s: str, p: str) -> list[int]:
    n, m = len(s), len(p)
    if n < m:
        return []
    need = Counter(p)
    window = Counter(s[:m])
    res = [0] if window == need else []
    for i in range(m, n):
        window[s[i]] += 1               # 右进
        window[s[i - m]] -= 1           # 左出
        if window[s[i - m]] == 0:
            del window[s[i - m]]        # 保持比较正确
        if window == need:
            res.append(i - m + 1)
    return res
```

复杂度：时间 O(n)，空间 O(字符集)。

最 Pythonic 写法：对每个定长切片建 `Counter` 与 `p` 比较，一行列表推导写完（O(n·k)，胜在极简）。

```
from collections import Counter

def findAnagrams(s: str, p: str) -> list[int]:
    need, m = Counter(p), len(p)
    return [i for i in range(len(s) - m + 1) if Counter(s[i:i + m]) == need]
```

---

10. 和为 K 的子数组（Subarray Sum Equals K，LC 560，Medium）

题意：求和恰为 `k` 的连续子数组个数（含负数）。

考点：为何不能用滑动窗口（有负数，单调性丧失）；前缀和 + 哈希。

思路：前缀和 `pre`，子数组 `(j, i]` 和为 `k` 等价于 `pre[i] - pre[j] = k`，即查 `pre - k` 出现次数。哈希记录前缀和频次。

```python
from collections import defaultdict

def subarraySum(nums: list[int], k: int) -> int:
    count = defaultdict(int)
    count[0] = 1            # 空前缀
    pre = 0
    res = 0
    for x in nums:
        pre += x
        res += count[pre - k]
        count[pre] += 1
    return res
```

复杂度：时间 O(n)，空间 O(n)。

最 Pythonic 写法：用 `accumulate` 生成前缀和、`Counter` 计数，省去手动累加。

```python
from collections import Counter
from itertools import accumulate

def subarraySum(nums: list[int], k: int) -> int:
    seen = Counter({0: 1})
    res = 0
    for pre in accumulate(nums):
        res += seen[pre - k]
        seen[pre] += 1
    return res
```